Branching

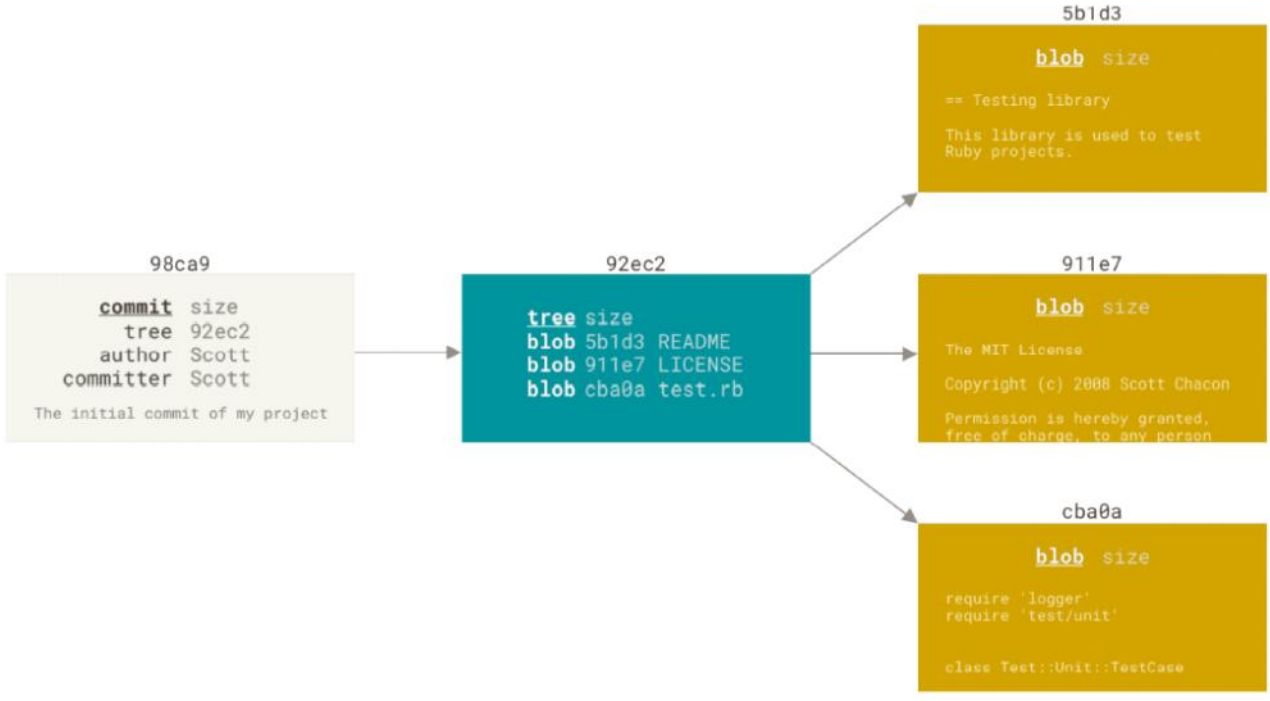
Tuesday, November 25, 2025 10:47 PM

Branching means you **diverge** from the **main line** of development and continue to do work without messing with that main line.

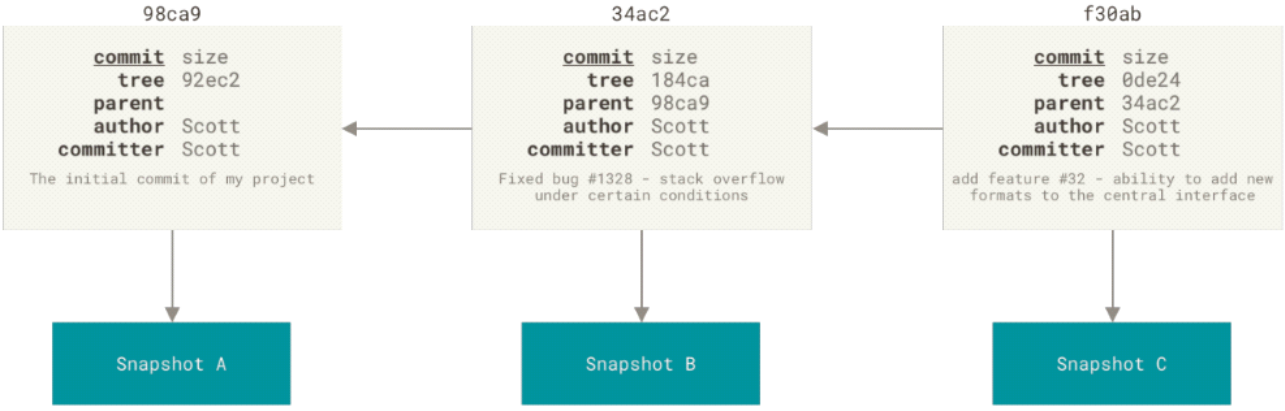
Let's examine, how Git **stores** its data. Git doesn't store data as a series of changesets or differences, but instead as a **series of snapshots**

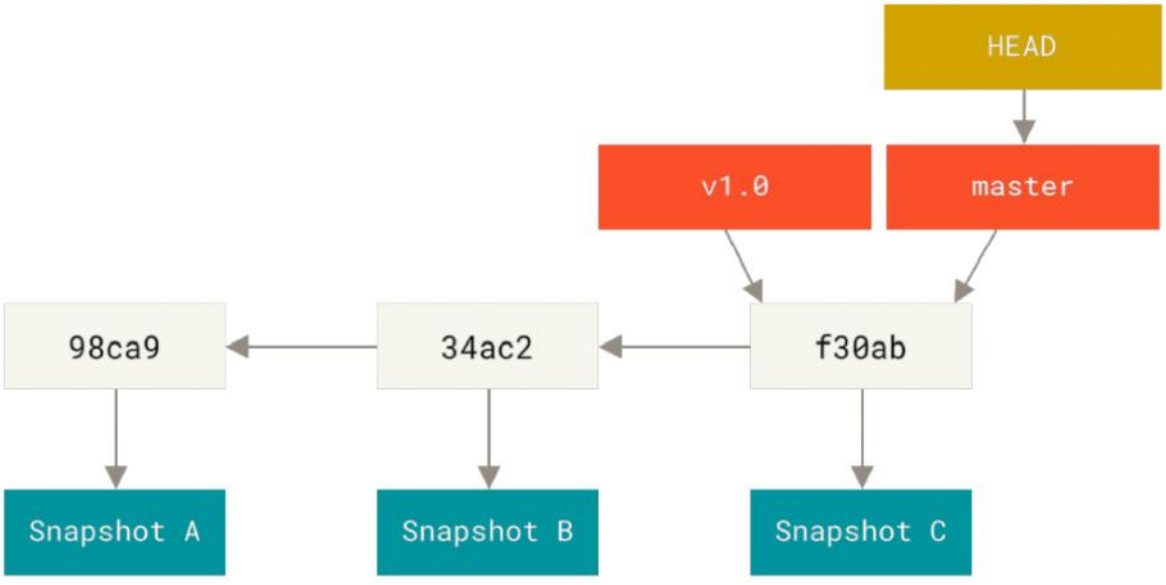
To visualize this, let's assume that you **stage** and **commit** *three* files in a directory.

1. Staging the files computes a **checksum** for **each one** (the SHA-1 hash),
2. Stores that version of the file in the Git repository (Git refers to them as blobs),
3. Adds that checksum to the staging area
4. On **commit**, Git **checksums** each **subdirectory** (in this case, just the root project directory) and stores them as a tree object in the Git repository
5. Git then **creates** a **commit object** that has the **metadata** and a **pointer** to the **root project** tree so it can re-create that snapshot when needed.
6. At this stage in our example, Git repository has **5 objects**
    a. **3 blobs** (each representing the contents of one of the three files)
    b. **1** tree blob that lists the contents of the directory and specifies which file names are stored as which blobs
    c. **1** commit with the pointer to that root tree and all the commit metadata



Now, If you **make** some **changes** and **commit again**, the next commit **stores** a **pointer** to the **commit** that came **immediately before** it.



What is a Branch

1. A branch in Git is simply a **lightweight movable pointer** to one of the **commits**.
2. The **default** branch **name** in Git is **master**.
3. As you start making commits, you're given a master branch that **points** to the **last commit** you made.
4. **Every time** you **commit**, the **master** branch pointer **moves** forward **automatically**